

UNIT 01 · CHAPTER 4

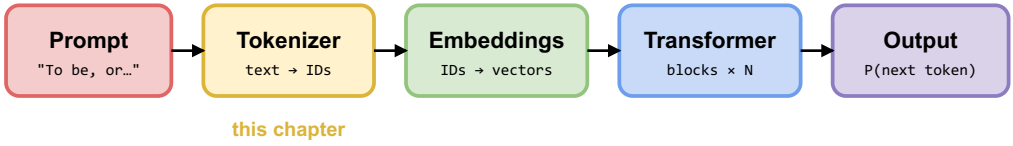
Tokenization

CSE 10124 · Building ChatGPT
William Theisen · University of Notre Dame

Text Input

Every input we have given a network so far has been a list of numbers. A Notre Dame game was a row of stats, and a picture of a handwritten digit is a `<28, 28>` grid of pixel brightnesses that we flatten into a `<1, 784>` vector. The network multiplies those numbers by its weights, adds its biases, and out comes a prediction, and optimization nudges the weights until the predictions get better.

This course builds a language model, and a language model's input is text. Unfortunately, our network has no idea what to do with "To be, or not to be". There is no way to multiply a word by a weight. **Before any part of the network can run, the text has to become numbers**, and the piece of a language model that does this is the **tokenizer**.



Pictured above is the whole pipeline of a model like ChatGPT. The tokenizer sits at the very start, it turns the prompt into a list of whole numbers, and everything after it only ever sees those numbers. It's also the simplest piece, and it isn't trained by gradient descent at all.

The Shakespeare Corpus

To build a tokenizer we first need text. The model we build in this class, **irishQwen** (a classroom version of Alibaba's Qwen3 models), learns from the complete works of Shakespeare: one plain text file from Project Gutenberg, every play and poem, `5,458,199` characters long. The collection of text a model learns from is called its **corpus**.

The whole file is a lot to look at, so for most of this chapter we'll work with one play out of it, Hamlet:

```
text = open('shakespeare.txt').read()
start = text.index('THE TRAGEDY OF HAMLET')
end = text.index('THE END', start)
hamlet = text[start:end]
print(len(text), len(hamlet))

i = hamlet.index('To be, or not to be')
print(hamlet[i:i + 88])
```

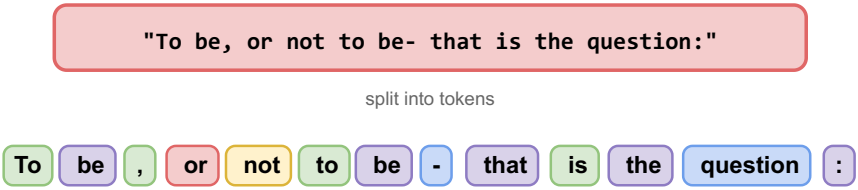
OUTPUT

```
5458199 196237
To be, or not to be- that is the question:
    Whether 'tis nobler in the mind to suffer
```

Hamlet alone is `196,237` characters. Our running example is the most famous line in it, exactly as this edition prints it (with a dash after the second "be", where other editions have a comma or a colon). Notice that the second line of verse starts with four spaces of indentation, which is also part of the text.

Tokens

The first decision is what the pieces should be. The obvious choice is words, so "To be, or not to be- that is the question:" becomes a list of words and punctuation marks:



Each piece is a **token**, one piece of text that the model treats as a single unit. Our line has `13`, ten words and three punctuation marks. In code, Python's `re` module can do the splitting:

```
1  import re
2  line = "To be, or not to be- that is the question:"
3  pattern = r" ?\w+| ?[^\w\s]|\s+"
4  tokens = re.findall(pattern, line)
5  print(tokens[:7])
6  print(tokens[7:])
```

OUTPUT

```
['To', ' be', ',', ' or', ' not', ' to', ' be']
['-', ' that', ' is', ' the', ' question', ':']
```

The pattern has three options: maybe a space then a run of letters, maybe a space then one punctuation mark, or a run of whitespace (newlines and indentation). Notice that **every word after the first keeps the space in front of it**, **be** rather than `"be"`. It looks odd, but no space is ever thrown away.

Vocabulary

`be` shows up twice in our line, and in a whole play it shows up hundreds of times. The model shouldn't have to learn about `be` once per appearance, so we keep a list of the distinct tokens, each written down once. This list is the **vocabulary**, the fixed collection of tokens a tokenizer can represent.

```
vocab = sorted(set(tokens))
print(vocab[:6])
print(vocab[6:])
```

OUTPUT

```
[To, be, ,, or, not, to, be]
[-, that, is, the, question, :]
[be, is, not, or, question, that]
[the, to, ,, -, :, To]
```

Our line's vocabulary has `12` tokens, one fewer than the line, since both `be`s share an entry. Sorting is just a convenient way to fix the order. Python sorts a space before any punctuation mark and punctuation before capital letters, which is why the words with a space in front come first and `To` comes last.

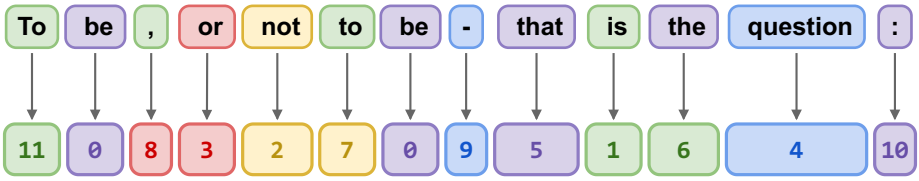
The vocabulary is fixed once it's built. From then on the tokenizer can only ever produce the tokens in it, and every token it produces is one of them.

Token IDs

Once the vocabulary is fixed, each token's place in it gives us a number. That number is the token's **token ID**, a whole number that labels one entry in the vocabulary. Here is our line's vocabulary with its IDs:

token	ID	token	ID
be	0	the	6
is	1	to	7
not	2	,	8
or	3	-	9
question	4	:	10
that	5	To	11

Replacing every token in the line by its ID gives `13` whole numbers:



Splitting text into tokens and then looking up each token's ID is called **tokenization**. The line is now **exactly the kind of input we know how to give a network**, a list of numbers, and the two be s are both `0`.

Encoding

Going from text to token IDs is called **encoding**. In code it's a dictionary that maps each token to its place in the vocabulary, and one lookup per token. The rest of the course is PyTorch, so the IDs go into a tensor:

```
import torch
token_to_id = {tok: i for i, tok in enumerate(vocab)}
ids = torch.tensor([token_to_id[t] for t in tokens])
print(ids)
```

OUTPUT

```
[To, be, ,, or, not, to, be]
[-, that, is, the, question, :]
[be, is, not, or, question, that]
[the, to, ,, -, :, To]
tensor([11,  0,  8,  3,  2,  7,  0,  9,  5,  1,  6,  4, 10])
```

(The block carries on from the two before it, so their output comes first.) The line is now a tensor of `13` IDs, one per token, and the two `be`s are both `0`. How long a piece of text is, counted in tokens, is going to matter a lot by the end of the chapter.

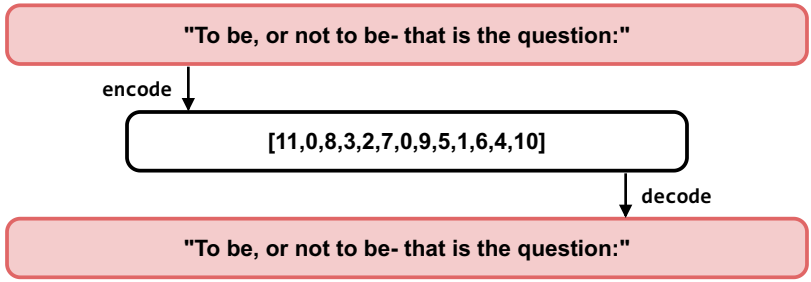
Decoding

A language model's answer comes out as token IDs too, so we also need to go back the other way. Reconstructing text from token IDs is called **decoding**: we flip the dictionary around, look up each ID, and glue the tokens back together.

```
id_to_token = {i: tok for tok, i in token_to_id.items()}
decoded = ''.join(id_to_token[i] for i in ids.tolist())
print(decoded)
print(decoded == line)
```

OUTPUT

```
[To, be, ,, or, not, to, be]
[-, that, is, the, question, :]
[be, is, not, or, question, that]
[the, to, ,, -, :, To]
tensor([11,  0,  8,  3,  2,  7,  0,  9,  5,  1,  6,  4, 10])
To be, or not to be- that is the question:
True
```



This is where keeping the spaces with the words pays off. **Decoding gives back exactly the text we encoded**, spaces and all. If we had thrown the spaces away we would get "Tobe,ornottobe-thatisthequestion:" back, and need some rule for guessing where the spaces go.

Larger Vocabularies

Nothing about encoding depends on the line being short. Here is the same tokenizer with a vocabulary built from all of Hamlet, used to encode our line:

```python
import re, torch
text = open('shakespeare.txt').read()
start = text.index('THE TRAGEDY OF HAMLET')
hamlet = text[start:text.index('THE END', start)]

pattern = r" ?\w+| ?[^\w\s]|\s+"
tokens = re.findall(pattern, hamlet)
vocab = sorted(set(tokens))
token_to_id = {tok: i for i, tok in enumerate(vocab)}
print(len(tokens), len(vocab))

line = "To be, or not to be- that is the question:"
ids = [token_to_id[t] for t in re.findall(pattern, line)]
print(torch.tensor(ids))
```

OUTPUT

```
47253 5865
tensor([5337,  975, 4854, 3192, 3115, 4366,  975, 4855, 4294, 2621, 4295,
```

Hamlet is `47,253` tokens long and its vocabulary has `5,865` entries. Our line gets much bigger IDs, since the vocabulary is much bigger (**be** was `0` and is now `975`), but **the same token always gets the same ID**, so both **be**s are still the same number.

The vocabulary grows with the text. One line needed `12` tokens, one play needs `5,865`, and the whole Shakespeare file `36,565`.

Token IDs Have No Meaning

It's tempting to read something into the IDs. Unfortunately they mean nothing at all. Here are a few more from Hamlet's vocabulary:

```
for t in [' king', ' queen', ' question', 'To', ' to']:
    print(repr(t), token_to_id[t])
```

OUTPUT

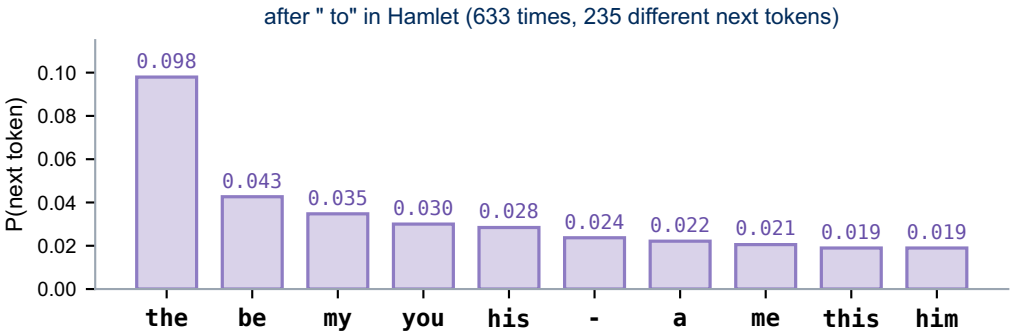
```
47253 5865
tensor([5337,  975, 4854, 3192, 3115, 4366,  975, 4855, 4294, 2621, 4295,
 king 2672
 queen 3552
 question 3554
To 5337
 to 4366
```

queen is 3552 and **question** is 3554, two apart, while **king** is 880 further away. They are neighbors only because they're spelled alike and sorting put them side by side. **To** and **to** are the same word to us and two unrelated numbers to the model.

So **comparing IDs does not measure meaning**. A token ID is a label, like the number on a football jersey: number 12 isn't a slightly better player than number 11. Turning IDs into something that does carry meaning is the job of the embeddings, a few chapters from now.

The Output of a Language Model

The vocabulary also decides what comes out the other end. A language model reads tokens and predicts the next one, and its prediction is **a probability for every token in the vocabulary**. It's the same softmax we used to classify digits, with one class per token instead of one per digit.



We don't have a trained model yet, but we can count. Above is what follows **to** in Hamlet: it appears `633` times, **the** comes next `62` of them (`0.098`) and **be** `27` (`0.043`), and `235` different tokens follow it at least once. A trained model's output looks like this, mostly small probabilities, one for each of the `5,865` tokens in the vocabulary.

So **the size of the vocabulary is the size of the model's output layer**. irishQwen with the real Qwen3-0.6B weights has a vocabulary of `151,936` tokens, and a comment in its code calls its last layer (`1,024` numbers in, one per token out) the most expensive matrix multiplication in the model.

Tokenizer Files

A vocabulary is built once and then used for everything after, training and every prompt anyone types, so it gets saved to a file. Almost every tokenizer is saved as **JSON** (JavaScript Object Notation), a plain text format for exactly the kind of dictionary we've been building. Python's `json` module writes it (we print only the start):

```
import re, json
line = "To be, or not to be- that is the question:"
pattern = r" ?\w+| ?[^\w\s]|\s+"
vocab = sorted(set(re.findall(pattern, line)))
token_to_id = {tok: i for i, tok in enumerate(vocab)}
print(json.dumps(token_to_id)[:56], '...')
```

OUTPUT

```
{ be: 0, is: 1, not: 2, or: 3, question: 4 ...
```

The JSON is just the dictionary written out as text, token then ID. Real tokenizer files look the same, only longer. The `tokenizer.json` of a model on Hugging Face holds a dictionary like this with every one of its tokens, well over a hundred thousand of them for Qwen3 or DeepSeek.

irishQwen's own Shakespeare tokenizer is saved the same way, in `Datasets/shakespeare_vocab.json`. Most of that file describes how its tokens are built, which is the next chapter's business, but near the top are three entries that aren't text at all.

Special Tokens

Here they are, as irishQwen's tokenizer file lists them (the first `256` IDs are taken, for reasons we'll see at the end of the chapter):

token	ID	stands for
<|pad|>	256	padding
<|sos|>	257	start of sequence
<|eos|>	258	end of sequence

These are **special tokens**, tokens added by hand for jobs that ordinary text can't do. The angle brackets and bars are there so they can never be confused with anything in Shakespeare. DeepSeek's file starts with the same three under longer names.

<|sos|> and <|eos|> mark the start and the end of a sequence. irishQwen's training script wraps the whole corpus in them before encoding it. The end token matters most when the model writes: **a model that can predict <|eos|> can say it's finished**, instead of producing Shakespeare forever.

<|pad|> is padding. A batch of sequences is one tensor, so every row has to be the same length, and shorter rows are filled out with the pad token. irishQwen is told which ID is padding, and leaves those positions out of the loss, so **padding never counts as text**.

Word-Level Tokenization

So far we've taken for granted that tokens are words (and punctuation marks). Splitting text this way is called **word-level tokenization**. It's the obvious choice, and it worked nicely on one line. On the whole Shakespeare file it gives `1,293,291` tokens of text and a vocabulary of `36,565`.

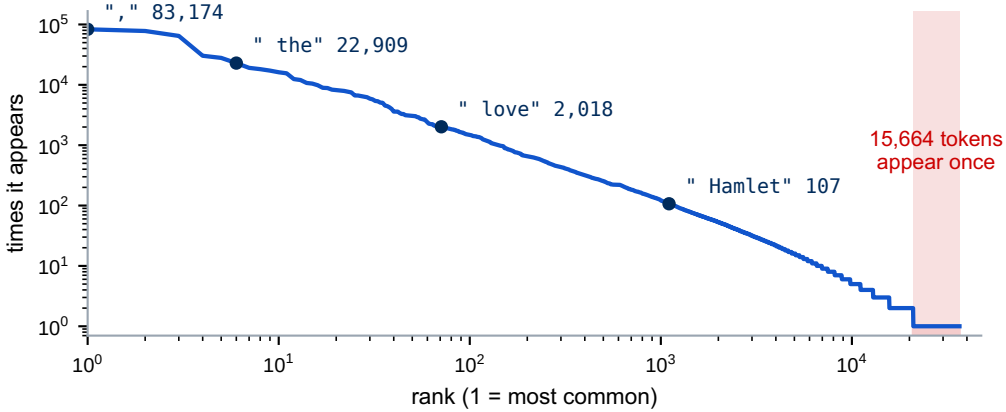
That vocabulary is big because every form of every word is its own token. Here are the most common tokens that start with **lov**:

token	times	token	times
love	2,018	lover	63
loves	189	lovers	54
lov	158	lovely	53
loving	115	loved	45
Love	76	lovest	7

There are `23` of them in all (**lov** on its own is Shakespeare's "lov'd", which our pattern splits at the apostrophe). To a word-level tokenizer **love**, **loves** and **Love** are as unrelated as **love** and "`turnip`". **Each one is a separate token the model has to learn about from scratch**, and each one is another entry in the output layer.

Rare Words

Worse, most of those tokens hardly ever appear. Pictured below is how often each of the `36,565` tokens appears in the whole file, most common first, on log scales:



A handful of tokens appear tens of thousands of times (`83,174` commas, `22,909` **the**s). Down the long tail on the right, `15,664` tokens, `43%` of the vocabulary, appear **exactly once** in all of Shakespeare.

A model can't learn much about a token from one example, and yet each of those `15,664` tokens still gets its own row of weights in the output layer. It turns out this shape (a few words used constantly, most words almost never) shows up in every language, so a bigger corpus doesn't make the tail go away. It just makes it longer.

Unknown Words

Rare words are a nuisance, but words that never appeared at all are worse. Here is a prompt a Notre Dame student might type, encoded with Hamlet's vocabulary:

```python
import re
text = open('shakespeare.txt').read()
start = text.index('THE TRAGEDY OF HAMLET')
hamlet = text[start:text.index('THE END', start)]
pattern = r" ?\w+| ?[^\w\s]|\s+"
vocab = sorted(set(re.findall(pattern, hamlet)))
token_to_id = {tok: i for i, tok in enumerate(vocab)}

prompt = "Great win last night irish!"
for t in re.findall(pattern, prompt):
    print(repr(t), token_to_id.get(t))
```

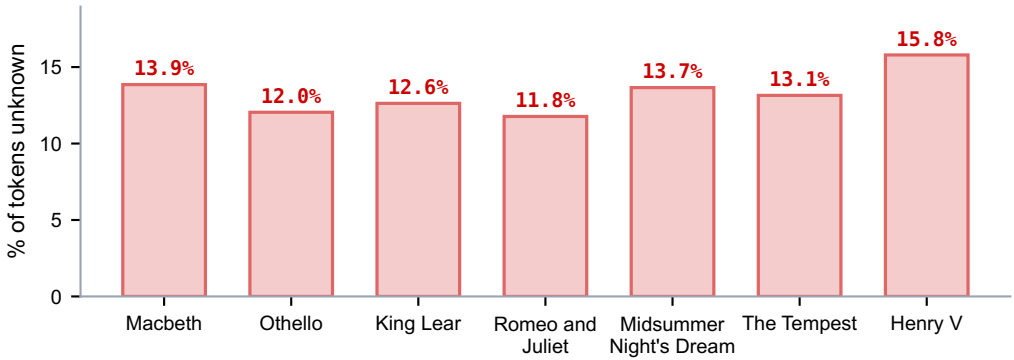
OUTPUT

```
'Great' None
 win 4732
 last 2718
 night 3091
' irish' None
! 4849
```

`dict.get` returns `None` for a token that isn't there. Hamlet never uses a capitalized "Great", and never mentions the Irish at all, so **a word-level tokenizer has no ID to give them**. Usually it has one more special token for this, written something like <|unk|>, and every unknown token becomes that, which keeps the code from crashing but throws the word away.

Out-of-Vocabulary Tokens

It isn't only modern slang. Hamlet's vocabulary misses `3,732` of the `26,938` tokens in Macbeth (`13.9%`), mostly names (**Macbeth** , **Banquo**) but also **Double** from "Double, double, toil and trouble", since Hamlet only has the lowercase **double** . Other plays do no better:



Tokens that aren't in the vocabulary are called **out-of-vocabulary** tokens. Henry V is the worst of these. Two of its scenes are mostly in French, but even without them it misses `14.7%`, still more than any other play here. Every new name, spelling or capital letter is a new token, so **a word-level vocabulary is never finished**, and a model that only knows Hamlet can't read about one token in eight of any other play.

Character-Level Tokenization

If words are too big, we can go the other way and make every single character a token. This is **character-level tokenization**. The code gets simpler, since there's no pattern, a string is already a list of characters:

```
import torch
text = open('shakespeare.txt').read()
start = text.index('THE TRAGEDY OF HAMLET')
hamlet = text[start:text.index('THE END', start)]

chars = sorted(set(hamlet))
char_to_id = {ch: i for i, ch in enumerate(chars)}
print(len(chars))

line = "To be, or not to be- that is the question:"
ids = torch.tensor([char_to_id[ch] for ch in line])
print(len(ids))
print(ids.view(3, 14))  # three rows of 14
```

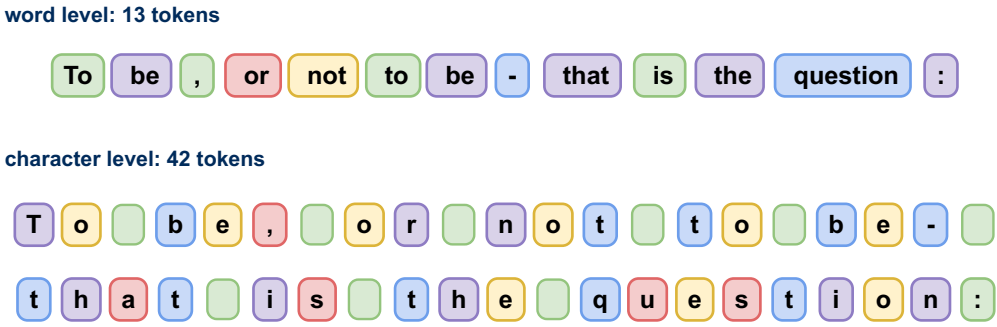
OUTPUT

```
74
42
tensor([[40, 62,  1, 49, 52,  8,  1, 62, 65,  1, 61, 62, 67,  1],
        [67, 62,  1, 49, 52,  9,  1, 67, 55, 48, 67,  1, 56, 66],
        [ 1, 67, 55, 52,  1, 64, 68, 52, 66, 67, 56, 62, 61, 16]])
```

Hamlet uses only **74** different characters, and the whole Shakespeare file **91** (both cases of the alphabet, the digits, punctuation, the space and the newline). **The vocabulary is tiny, and it's finished**: "irish" is just five letters we already have, so there are no unknown tokens.

Sequence Length

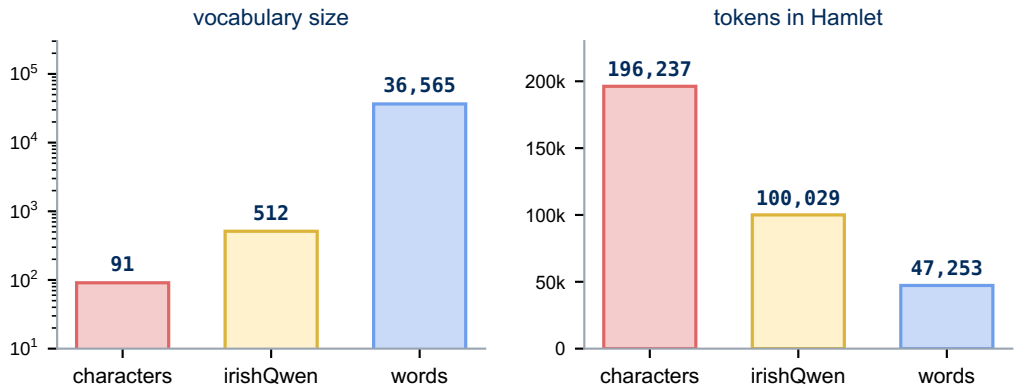
Unfortunately, the line that was `13` tokens is now `42`, and Hamlet goes from `47,253` word-level tokens to `196,237` character-level ones, about four times as many. The number of tokens a piece of text becomes is its **sequence length**.



Sequence length matters because a language model reads a fixed number of tokens at a time, its **context window**. irishQwen trains on Shakespeare with a window of `128` tokens. Starting from our line, `128` characters reach "The slings and arrows of outrageous" and stop before "fortune", while `128` word-level tokens reach "what dreams may come", about ten lines later. **The same window sees four times less text** when its tokens are characters.

Vocabulary Size vs. Sequence Length

Every token is also one more trip through the whole network, so four times as many tokens is about four times the work for the same text. Put side by side, the two levels are opposites:



Words give a huge vocabulary (a huge output layer, rare tokens, unknown tokens) but short sequences. Characters give a tiny vocabulary with nothing unknown, but sequences four times as long, made of tokens that mean very little on their own (`q` tells the model almost nothing). **Making the vocabulary smaller makes every text longer**, and the other way around.

The yellow bars in the middle are irishQwen's own tokenizer. Its vocabulary has `512` tokens, and it turns Hamlet into `100,029` tokens, between the two levels on both counts. It gets there by using pieces bigger than characters but (mostly) smaller than words.

Subwords

Look back at the **lov** table and much of it repeats. **loves**, **loved**, **lover** and **loving** are all **lov** plus a common ending. The endings repeat across words too: `2,270` different word-level tokens in Shakespeare end in "ing" (used `13,095` times between them), and `198` lowercase ones end in Shakespeare's "eth" ("loveth", "cometh").

The lecture's example sentence, "The cat sat on the mat and ate.", has the same thing on a small scale: "at" turns up inside "cat", "sat", "mat" and "ate". A **subword** is a reusable piece smaller than a whole word, like **lov**, **ing** or **at**.

With a vocabulary of subwords, a few hundred pieces can spell every word in the love family, **the common words can still be whole tokens**, and a word nobody has seen before can be spelled from smaller pieces, in the worst case one character at a time.

Subword Tokenizers

This is how irishQwen's tokenizer splits our line (dots mark the spaces):



Common words like **not**, **that** and **the** are single tokens, and `" question"`, which is rarer, is spelled from four pieces: a space, **qu**, **est** and **ion**. The line is 17 tokens, between the 13 words and the 42 characters.

The first 256 IDs in irishQwen's vocabulary are single bytes, which for plain English text means single characters, so **it can spell anything**. "Great win last night irish!" becomes `G re at ·w in ·l ast ·n ight · ir is h !`, no unknown tokens and 14 tokens instead of 27 characters. Essentially every large language model works this way. OpenAI's tokenizer page splits the same prompt into 7 tokens for its GPT-5.x models, which have a far bigger vocabulary.

Which leaves the question of where pieces like **qu**, **est** and **the** came from. Nobody chose irishQwen's 253 pieces by hand (its 512 tokens are the 256 bytes, the 3 special tokens and those 253). They were learned from Shakespeare by counting, with an algorithm called **Byte Pair Encoding**.